arXiv:2506.13249v1 [cs.AI] 16 jun 2025

Busca em Árvore Monte-Carlo com Números de Prova Generalizada

(Generalized Proof-Number Monte-Carlo Tree Search)

Jakub Kowalski[a,*], Dennis J. N. J. Soemers[b], Szymon Kosakowski[a] e Mark H. M. Winands[b]

[a]University of Wrocław

[b]Maastricht University

ORCID (Jakub Kowalski): https://orcid.org/0000-0003-1932-4278, ORCID (Dennis J. N. J. Soemers): https://orcid.org/0000-0003-3241-8957, ORCID (Szymon Kosakowski): https://orcid.org/0009-0006-7992-0310, ORCID (Mark H. M. Winands): https://orcid.org/0000-0002-0125-0824

[*] Autor correspondente. E-mail: jakub.kowalski@cs.uni.wroc.pl

Resumo. Este artigo apresenta a Busca em Árvore Monte-Carlo com Números de Prova Generalizada (*Generalized Proof-Number Monte-Carlo Tree Search*): uma generalização de combinações recentemente propostas da Busca por Números de Prova (PNS) com a Busca em Árvore Monte-Carlo (MCTS), que usam números de (des)prova para enviesar estratégias de Seleção baseadas em UCB1 em direção a partes da busca que se espera serem provadas (ou refutadas) com facilidade. Propomos três modificações centrais às combinações anteriores de PNS com MCTS. Primeiro, rastreamos números de prova por jogador. Isso reduz a complexidade do código, no sentido de que não precisamos mais de números de refutação, e generaliza a técnica para jogos com mais de dois jogadores. Segundo, propomos e avaliamos extensivamente diferentes métodos de usar números de prova para enviesar a estratégia de seleção, obtendo forte desempenho com estratégias mais simples de implementar e calcular. Terceiro, combinamos nossa técnica com a MCTS com Limites de Pontuação (*Score Bounded MCTS*), permitindo que o algoritmo prove e utilize limites superiores e inferiores de pontuação — em vez de apenas provar vitórias ou não-vitórias. Os experimentos demonstram aumentos substanciais de desempenho, chegando à faixa de 80% em 8 dos 11 jogos de tabuleiro testados.

1 Introdução

A Busca em Árvore Monte-Carlo (MCTS) [10, 13] é um método de busca *best-first* guiado pelos resultados de simulações de Monte-Carlo, bem estabelecido em IA para jogos [7, 28]. Usando os resultados de simulações anteriores, o método constrói gradualmente uma árvore de jogo na memória e se torna cada vez melhor em estimar com precisão os valores dos movimentos mais promissores. A MCTS avançou substancialmente o estado da arte em vários domínios de jogos determinísticos, em particular o Go [23], mas também em outros jogos de tabuleiro, incluindo Amazons [16], Hex [2], Lines of Action [27] e Jogos Gerais (*General Game Playing*, GGP) [5].

Em jogos táticos, nos quais a linha principal rumo à posição vencedora costuma ser estreita, com muitas alternativas que não progridem, a MCTS pode frequentemente levar a um resultado errôneo, porque os valores dos nós na árvore não convergem rápido o bastante para o seu valor teórico-jogo. Para mitigar esse efeito, foram propostas variantes de MCTS que integram busca minimax [26, 25, 15, 4]. Recentemente, a Busca por Números de Prova (PNS) [1] foi integrada à MCTS [11, 20]. A PNS tem a vantagem de provar finais de jogo mais rápido que o minimax tradicional em muitos domínios. Mostrou-se que a variante PN-MCTS [14] melhora a MCTS padrão em domínios como Lines of Action, MiniShogi, Knightthrough e Awari.

Neste artigo, propomos uma generalização da PN-MCTS, chamada Busca em Árvore Monte-Carlo com Números de Prova Generalizada (*Generalized Proof-Number Monte-Carlo Tree Search*, GPN-MCTS). Primeiro, essa extensão da PN-MCTS rastreia números de prova por jogador, o que reduz a complexidade do código, no sentido de que não precisamos mais de números de refutação, e generaliza a técnica para jogos com mais de dois jogadores. Segundo, a GPN-MCTS contém diferentes melhorias no uso de números de prova para enviesar a estratégia de seleção UCT [13], obtendo forte desempenho com estratégias mais simples de implementar e calcular. Terceiro, a PN-MCTS é integrada à MCTS com Limites de Pontuação (*Score Bounded MCTS*) [8], permitindo que a técnica prove e utilize limites superiores e inferiores de pontuação — em vez de apenas provar vitórias ou não-vitórias.

Para os experimentos, a GPN-MCTS foi implementada no sistema de jogos gerais Ludii [19].[1] Embora não haja garantia de que funcione para todo jogo, em muitos casos em que funciona suas melhorias são significativas,

[1]Após o processo de revisão, um *pull request* será proposto para incluir a GPN-MCTS no repositório público oficial do projeto Ludii.

chegando a 80% de taxa de vitória em 8 dos 11 jogos de tabuleiro testados e a mais de 60% nos três restantes. Além disso, mostramos que ela combina bem com o aprimoramento Score-Bounded MCTS.

Este artigo está organizado da seguinte forma. Primeiro, a Seção 2 descreve a base metodológica deste trabalho, a MCTS e a PNS. Em seguida, a MCTS com Números de Prova Generalizada é apresentada na Seção 3. Depois, a abordagem é testada em um conjunto diversificado de jogos na Seção 4. Por fim, o artigo conclui e apresenta perspectivas de pesquisas futuras na Seção 5.

2 Fundamentação

2.1 Busca em Árvore Monte-Carlo

A Busca em Árvore Monte-Carlo (MCTS) [10, 13] é um método de busca *best-first* que constrói gradualmente uma árvore de busca, equilibrando o aproveitamento (*exploitation*) de partes que parecem promissoras com base em iterações anteriores com a exploração (*exploration*) de partes pouco visitadas. Ela faz isso iterando por quatro etapas estratégicas [9], ilustradas na Figura 1, até que o orçamento de tempo ou de iterações se esgote.

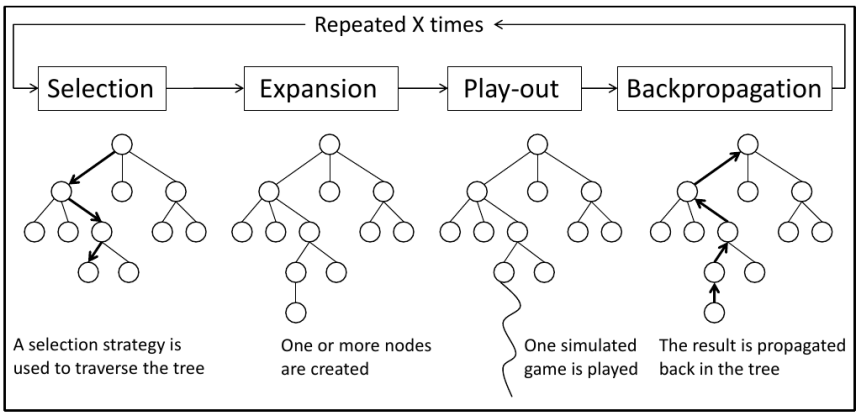


Figura 1: Visão geral da Busca em Árvore Monte-Carlo.

Etapa de Seleção. A etapa de seleção percorre a árvore, a partir do nó raiz, até alcançar um nó para o qual ainda existam ações legais que não foram expandidas em nós da árvore de busca (ou até alcançar um nó terminal). Essa etapa implementa o equilíbrio entre exploração e aproveitamento. Uma das implementações-base mais comuns da MCTS — chamada de *Upper Confidence Bounds applied to Trees* (UCT) [13] — usa a estratégia UCB1 [3] para escolher entre os filhos de qualquer nó atual. Ela funciona da seguinte forma. Seja I o conjunto de nós imediatamente alcançáveis a partir do nó atual p. A estratégia de seleção escolhe o filho b do nó p que satisfaz a Fórmula (1):

$$b \in \arg\max_{i \in I} \left(v_i + C \times \sqrt{\frac{\ln n_p}{n_i}} \right), \tag{1}$$

onde v_i é o valor estimado do nó i, n_i é a contagem de visitas de i e n_p é a contagem de visitas de p. C é um hiperparâmetro que pode ser ajustado experimentalmente. Aqui, os empates são desfeitos aleatoriamente.

Etapa de Expansão. Como dito anteriormente, a etapa de seleção continua até que se alcance um nó que ainda não teve todos os seus filhos expandidos. Entre os filhos que ainda não foram armazenados na árvore, um é selecionado uniformemente ao acaso. Esse nó L é então adicionado como um novo nó folha. Se a etapa de seleção chega a um nó terminal, a expansão e as etapas de simulação subsequentes são puladas.

Etapa de Simulação (Play-out). A partir do novo nó folha L, executa-se a etapa de simulação. Os movimentos são escolhidos em autojogo até que o fim do jogo seja alcançado. Essa etapa pode consistir em jogar movimentos uniformemente aleatórios ou — frequentemente melhor — movimentos semialeatórios escolhidos de acordo com uma *estratégia de simulação*.

Etapa de Retropropagação (Backpropagation). Na etapa final, o resultado R de uma simulação k é retropropagado a partir do nó folha L, passando pelos nós previamente percorridos, até chegar à raiz. O resultado é pontuado positivamente ($R_k = +1$) se o jogo é vencido e negativamente ($R_k = -1$) se o jogo é perdido. Empates levam a um resultado $R_k = 0$. Uma *estratégia de retropropagação* é aplicada ao *valor* v_i de um nó i. Aqui, ele é calculado

tomando a média dos resultados de todos os jogos simulados que passaram por esse nó [10], isto é, $v_i = (\Sigma_{k \in K} R_k)/n_i$, em que K é o conjunto de índices de todas as simulações. As contagens de visitas n_i de todos os nós ao longo da trajetória também são incrementadas.

Quando o orçamento de busca se esgota, o movimento que acaba sendo jogado é aquele, a partir do nó raiz, que tem a maior contagem de visitas (embora outras estratégias também sejam possíveis [9]).

2.2 Busca por Números de Prova

A Busca por Números de Prova (PNS) é um método de busca *best-first* especialmente adequado para encontrar o valor teórico-jogo em árvores de jogo [1]. Seu objetivo é provar uma meta específica. No contexto deste artigo, a meta é provar uma vitória forçada para o jogador da vez. Uma árvore pode ter três valores: *verdadeiro*, *falso* ou *desconhecido*. No caso de uma vitória forçada, a árvore é *provada* e seu valor é verdadeiro. No caso de uma derrota ou empate forçado, a árvore é *refutada* e seu valor é falso. Caso contrário, o valor da árvore é desconhecido. Enquanto o valor da raiz for desconhecido, o nó mais promissor é expandido. Assim como a MCTS, a PNS não precisa de uma função de avaliação heurística dependente do domínio para determinar o nó mais promissor [1]. Na PNS, esse nó costuma ser chamado de nó *mais provante* (*most-proving node*). A PNS seleciona o nó mais provante usando dois critérios: (1) o formato da árvore de busca (o fator de ramificação de cada nó interno) e (2) os valores das folhas. Esses dois critérios permitem que a PNS trate árvores de jogo com fator de ramificação não uniforme de modo eficiente.

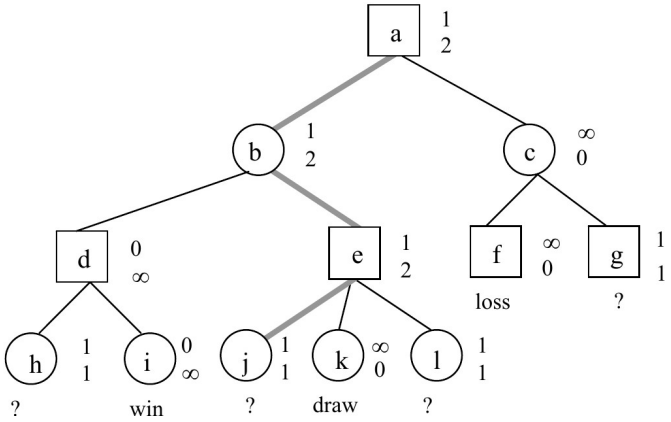


Figura 2: Uma árvore AND/OR com números de prova e de refutação. Um quadrado denota um nó OR e um círculo denota um nó AND. Os números à direita de um nó denotam o número de prova (superior) e o número de refutação (inferior).

A busca PN representa o jogo como uma árvore AND/OR. Os nós OR correspondem a posições em que o primeiro jogador joga, enquanto nos nós AND quem joga é o segundo jogador. Um exemplo dessa árvore é dado na Fig. 2. Na PNS, o *número de prova* (*pn*) representa o número mínimo de nós folha que precisam ser provados para provar o nó. De forma análoga, um *número de refutação* (*dpn*) representa o número mínimo de nós folha que precisam ser refutados para refutar o nó. Como a meta da busca é provar uma vitória forçada, os nós vencedores são considerados provados. Portanto, eles têm pn = 0 e dpn = ∞. Os nós perdidos ou empatados são considerados refutados. Eles têm pn = ∞ e dpn = 0. Os nós folha desconhecidos têm pn = 1 e dpn = 1. O pn de um nó OR interno é igual ao mínimo dos números de prova de seus filhos, pois, para provar um nó OR, basta provar um filho. O dpn de um nó OR interno é igual à soma dos números de refutação de seus filhos, pois, para refutar um nó OR, todos os filhos precisam ser refutados. O pn de um nó AND interno é igual à soma dos números de prova de seus filhos, pois, para provar um nó AND, todos os filhos precisam ser provados. O dpn de um nó AND é igual ao mínimo dos números de refutação de seus filhos, pois, para refutar um nó AND, basta refutar um filho.

O procedimento para selecionar o nó mais provante a ser expandido em seguida é o seguinte. O algoritmo começa na raiz. Então, em cada nó OR, o filho com o menor pn é selecionado como sucessor e, em cada nó AND, o filho com o menor dpn é selecionado como sucessor. Por fim, quando uma folha é alcançada, ela é expandida (o que transforma a folha em um nó interno) e os filhos recém-criados são avaliados.

2.3 Busca em Árvore Monte-Carlo com Números de Prova

A Busca em Árvore Monte-Carlo com Números de Prova (PN-MCTS) foi proposta inicialmente em [11], introduzindo um aprimoramento da fórmula UCB1 para jogos de soma zero com dois jogadores que inclui um termo relacionado à PNS, enviesando a seleção em direção aos filhos preferidos pela Busca por Números de Prova. Ela foi

posteriormente estendida por [14] para aproveitar a observação de que os números de (des)prova já calculados também podem servir para enviesar a seleção final de movimento e para pular subárvores resolvidas, de modo semelhante à Score-Bounded MCTS [8]. Neste trabalho, buscamos generalizar todas as contribuições acima.

A PN-MCTS rastreia números de prova e de refutação do ponto de vista do jogador da raiz em todos os nós da árvore MCTS. Os nós OR/AND são atribuídos de acordo com isso, a partir da perspectiva do jogador da raiz. A parte do algoritmo baseada na PNS é do tipo paranoico [22], pois as decisões do oponente, baseadas nos números de refutação, fazem com que ele esteja mais interessado em impedir a vitória do jogador da raiz do que em maximizar seu próprio resultado.

A fórmula UCT-PN proposta, mostrada em (2), estende a UCB1 padrão (1), introduzindo um termo que é maior para os nós filhos que estão próximos de serem resolvidos de acordo com os números de (des)prova, ponderado por uma constante C_{pn}.

$$b \in \arg\max_{i \in I} \left(v_i + C \times \sqrt{\frac{\ln n_p}{n_i}} + C_{pn} \times \mathrm{PNRank}(i, I) \right) \tag{2}$$

Em vez de usar diretamente os números de (des)prova na fórmula, a ideia é ordenar seus valores e usar um sistema de ranqueamento. Esse processo busca refletir a observação de que as magnitudes das diferenças entre os números de (des)prova tecnicamente não têm muito significado, e é encapsulado pela função PNRank (3).

$$\mathrm{PNRank}(i, I) = 1 - \frac{\mathrm{rank}(i)}{\max_{j \in I} \mathrm{rank}(j)} \tag{3}$$

O melhor nó — aquele que seria escolhido pela PNS — recebe posto 1. Esse é o filho com o menor número de prova nos nós OR e o com o menor número de refutação nos nós AND. O próximo na ordem receberia posto 2, e assim por diante. Nós empatados recebem o mesmo posto. Em seguida, os postos são normalizados para o intervalo [0, 1], para facilitar o escalonamento com os termos de exploração e aproveitamento da fórmula UCB1.

Os experimentos realizados em [14] mostraram que a introdução da UCT-PN foi suficiente para atingir um desempenho esmagador (cerca de 87% contra a MCTS pura) em Lines of Action. Extensões adicionais, como a seleção final de movimento e a resolução de subárvores, foram necessárias para obter taxas de vitória acima de 65% em MiniShogi e Knighttrough. Para obter uma taxa de vitória favorável no último dos jogos testados, o Awari, foi introduzida uma abordagem especial para tratar empates, envolvendo a manutenção de um segundo conjunto de números de (des)prova.

3 MCTS com Números de Prova Generalizada

Nesta seção, apresentamos o algoritmo GPN-MCTS e discutimos suas principais características.

3.1 Números de Prova por Jogador

O objetivo da fórmula PN-UCB é enviesar a exploração em direção a subárvores que podem ser provadas rapidamente. Assim, uma suposição razoável é que cada jogador está interessado em sua própria vitória. Como mencionado antes, essa não é a abordagem adotada pela versão da PN-MCTS descrita em [14]. Naquele trabalho, todos os números de (des)prova são calculados do ponto de vista do jogador da raiz, com os números de prova sendo usados para guiar a seleção nos nós OR e os números de refutação nos nós AND. Essa abordagem resulta em assimetria de comportamento, particularmente em jogos nos quais podem ocorrer empates, já que o jogador da raiz tenta provar suas vitórias, ao passo que o oponente apenas tenta refutar as vitórias do jogador da raiz (isto é, tenta provar qualquer mistura de empates e vitórias para si).

Propomos uma abordagem diferente, que consiste em rastrear números de prova separados por jogador em cada nó da árvore de busca. Isso reduz a complexidade do código, no sentido de que não precisamos mais codificar uma lógica separada para tratar números de refutação, e o comportamento da busca se torna mais simétrico, já que cada jogador tenta provar suas próprias vitórias.

Essa modificação influencia a forma como o algoritmo percebe os nós de prova AND/OR. Seja i um nó da árvore de jogo e p o jogador da vez nesse nó. Para provar uma vitória, basta que qualquer um dos caminhos disponíveis leve a uma vitória; portanto, esse é um nó OR para o jogador p. Da perspectiva de quaisquer jogadores que não p, o mesmo

nó é tratado como um nó AND, no qual todos os filhos precisam ser provados para que o nó seja considerado provado.

A vantagem da abordagem com um único número de prova por jogador é que ela se generaliza naturalmente para jogos com mais de dois jogadores. Embora as árvores de números de prova deixem de ser compostas por camadas alternadas de AND/OR, a forma como o algoritmo se comporta permanece inalterada. Para cada jogador, ao calcular sua árvore de prova, os nós do primeiro jogador são nós OR e todos os demais são nós AND.

Dessa forma, o algoritmo ainda supõe que todo jogador está interessado em provar a sua própria vitória. No entanto, provar uma vitória para um dado jogador ainda exige uma suposição paranoica sobre as estratégias de seus oponentes. Assim, nesta parte, nosso algoritmo se comporta como a Busca por Números de Prova Paranoica (*Paranoid Proof-Number Search*) [22], uma generalização da Busca por Números de Prova que permite provar jogos com qualquer número de jogadores.

3.2 Fórmulas UCT-PN

O propósito do termo PN na fórmula UCT-PN é enviesar a busca em direção a nós potencialmente mais fáceis de provar. No entanto, a força real desse viés, e como ele depende dos números de prova efetivos, é um ponto crítico a discutir. A Fórmula (3), que usa a ordenação dos valores por postos, foi proposta por [11, 14] com a justificativa de que as magnitudes das diferenças entre os números de (des)prova não são significativas.

Suspeitamos que descartar a informação sobre os valores reais de (des)prova seja um possível desperdício e decidimos buscar alternativas. De fato, acreditamos que existam muitas funções que poderiam cumprir esse papel e que podem ser mais simples de implementar e computacionalmente mais baratas do que a PNRank. Isso pode ser de especial importância, pois o problema da fórmula PNRank é a sua ineficiência. A necessidade de ordenar os valores de todos os filhos toda vez que um número de (des)prova é atualizado, para calcular o posto correto, é um esforço computacional significativo. Note que esse comportamento padrão pode ser otimizado: seja aproveitando a ordenação parcial e corrigindo apenas a posição do elemento atualizado (em $O(|I|)$), seja usando um método descrito na Seção 3.2.5.

Neste artigo, introduzimos duas fórmulas alternativas que podem ser usadas no lugar da anterior: PNMax e PNSum. Acreditamos que ambas representam abordagens naturais ao tentar levar em conta os valores de prova reais e distribuir o efeito de forma proporcional, e não uniforme.

3.2.1 Propriedades esperadas

Começamos estabelecendo as propriedades desejadas de uma função de viés baseada em números de prova, para que ela se encaixe bem no arcabouço e seu tipo possa ser tratado como um parâmetro da GPN-MCTS.

Assim, de acordo com a estrutura de (3), a fórmula adequada deve ser uma função que recebe um nó i e um conjunto de nós I (sendo i e todos os seus irmãos) e retorna um número em [0, 1], com valores maiores representando um viés maior em direção à seleção do nó.

Além disso, definimos um conjunto de condições que, em nossa opinião, uma fórmula de viés adequada deve cumprir. Seja pn(i) o número de prova de um nó i.

(a) A fórmula deve retornar 0 se for impossível provar um nó (pn(i) é ∞).

(b) Se, para algum $j \in I$, pn(j) é infinito, os valores de todos os filhos finitos devem ser estritamente positivos.

(c) Se pn(i) é igual a pn(j), então a saída para i e j, dado o mesmo I, também deve ser igual.

(d) (opcional) A fórmula deve sempre retornar 1 para o menor valor finito de pn(i) em I.

As invariantes acima garantem que o comportamento real da fórmula de viés esteja alinhado com as expectativas naturais. Para a fórmula PNRank definida em (3), as três primeiras condições são satisfeitas.

3.2.2 PNMax

A primeira das abordagens recém-propostas é escalar os valores em relação ao intervalo dos valores de prova (finitos) entre os filhos. Essa fórmula de viés, chamada PNMax (4), requer calcular os números de prova mínimo e máximo entre todos os filhos, o que é significativamente menos custoso computacionalmente do que ordenar.

As funções $\max_f$ e $\min_f$ calculam, respectivamente, o máximo e o mínimo dos valores de prova no conjunto de nós dado, levando em conta apenas valores finitos. Assim, $\max_f(I) = \max_{j \in I \wedge pn(j) \neq \infty} pn(j)$.

$$\text{PNMax}(i,I) = \begin{cases} 0 & \text{se } pn(i) \text{ é } \infty \\ 1 - \dfrac{pn(i) - \min_f(I)}{1 + \max_f(I) - \min_f(I)} & \text{caso contrário} \end{cases} \tag{4}$$

Para essa fórmula, todas as quatro condições são satisfeitas.

3.2.3 PNSum

Como alternativa, propomos uma fórmula de viés que distribui os valores proporcionalmente em relação à soma de todos os valores de prova finitos entre os filhos. Para essa fórmula, PNSum (5), todas as condições, exceto (d), são atendidas.

$$\text{PNSum}(i,I) = \begin{cases} 0 & \text{se } pn(i) \text{ é } \infty \\ 1 - \dfrac{pn(i)}{1 + \sum_{j \in I \wedge pn(j) \neq \infty} pn(j)} & \text{caso contrário} \end{cases} \tag{5}$$

A propriedade singular da PNSum é que seu comportamento está muito mais fortemente correlacionado com o fator de ramificação do nó pai do que nas outras fórmulas testadas. Isso, dependendo do jogo e do cenário de teste, pode se revelar um benefício ou uma desvantagem.

3.2.4 Ajuste da constante Cpn

Pesquisas anteriores sobre a PN-MCTS foram, de certa forma, baseadas na suposição de que a constante C_{pn} é universal e que, tal como a constante C da MCTS, o mesmo valor pode ser usado em uma variedade de jogos. Assim, a relação entre C_{pn} e cada jogo testado não foi devidamente estudada. Como nossa abordagem introduz funções alternativas para servir como fórmulas UCT-PN, a necessidade de ajuste por jogo e por fórmula parece ainda mais importante.

Embora os experimentos e a discussão associada sejam apresentados na Seç. 4.1, julgamos válido afirmar aqui, como parte essencial da descrição do algoritmo, que o valor de C_{pn} de fato tem um impacto crucial no desempenho da GPN-MCTS. Além disso, o desempenho máximo pode ocorrer em qualquer ponto do espectro de valores testados, e o pico para diferentes fórmulas UCT-PN pode ser atingido por valores de C_{pn} diferentes para o mesmo jogo.

3.2.5 Otimização da retropropagação

As três fórmulas de viés UCT-PN compartilham a mesma característica: seu valor para um dado nó depende de todos os filhos do mesmo pai. No caso de PNMax e PNSum, esses são fatores que podem ser associados ao nó pai e simplesmente armazenados nele como valores pré-calculados, mas, para a PNRank, a fórmula é mais complicada e se beneficia fortemente do cálculo do valor UCT-PN de todos os filhos de uma só vez.

Isso, no entanto, complica a etapa usual de retropropagação da PNS. Originalmente, ela é uma operação barata, que exige a atualização do valor de um pai se o valor de algum filho foi modificado. Agora, além do número de prova, também precisamos levar em conta um valor UCT-PN e, portanto, todos os filhos do pai devem ser atualizados, o que é significativamente mais caro, especialmente para a PNRank.

Contudo, podemos pelo menos mitigar parcialmente esse custo. A observação principal é que nem todos os valores que seriam alterados durante a retropropagação serão de fato requisitados pela UCT-PN durante a etapa de seleção da MCTS. Assim, em nossa implementação, otimizamos essa etapa definindo a flag *needRecalc* em um nó com número de prova atualizado, sem pré-calcular o valor UCT-PN para esse pai nem para seus irmãos. A fórmula UCT-PN é calculada sob demanda (*call-by-need*), somente se um nó com *needRecalc* for encontrado durante a fase de seleção (e, depois disso, a flag é definida como falsa).

3.3 Inicialização baseada em mobilidade

A maioria das melhorias clássicas da PNS se concentra em tratar o problema do consumo de memória ao tentar resolver a árvore de jogo [6, 17], de modo que não têm aplicação quando os números de prova são usados como viés de seleção dentro do algoritmo de jogo.

No entanto, o aprimoramento de inicialização por mobilidade [24] pode ser aplicado de forma direta à GPN-MCTS. A ideia é inicializar os nós folha desconhecidos de uma forma mais elaborada do que a descrita na Seção 2.2. Em um nó AND, o número de prova pode ser definido como o número de movimentos legais nesse nó. (Na PNS, os números

de refutação são inicializados dessa maneira nos nós OR.) Essa otimização melhora a qualidade dos números de (des)prova, pois funciona como uma antecipação de um passo para calcular os tamanhos estimados das subárvores a serem provadas.

3.4 GPN-MCTS com Limites de Pontuação

Em implementações práticas, algoritmos como a MCTS quase nunca são aplicados em sua forma pura. Em geral, o algoritmo resultante consiste na união de alguns aprimoramentos gerais com melhorias dependentes do jogo. Por essa razão, um de nossos objetivos foi testar a GPN-MCTS proposta usando uma configuração de algoritmo avançada e bem estabelecida e ver se os resultados melhorados se mantêm. Decidimos adotar como linha de base uma versão da MCTS com dois aprimoramentos. Um é simplesmente a técnica padrão de reutilização de árvore [7] e o outro é a MCTS com Limites de Pontuação [8].

A MCTS com Limites de Pontuação (SB MCTS) é uma extensão da MCTS Solver [26] que consegue lidar com empates e é generalizada para jogos com muitos resultados possíveis, bem como para jogos com mais de dois jogadores. A resolução de subárvores é provavelmente a única linha de aprimoramentos da MCTS que pode ser considerada obrigatória, pois quase não tem desvantagens. É fácil de implementar, tem custo computacional desprezível e, na maioria das vezes, aumenta a força de jogo de um agente quando aplicada a jogos de soma zero.

A Busca por Números de Prova e a MCTS com Limites de Pontuação baseiam-se em uma premissa semelhante; em ambos os casos, estamos rastreando quais subárvores da árvore de busca estão resolvidas. Por um lado, a PNS armazena dados que lhe permitem prever quais subárvores podem ser resolvidas com menos esforço, informação que falta à SB MCTS. Por outro lado, a SB MCTS requer apenas dois valores (limite inferior, limite superior) por nó da árvore para armazenar retornos garantidos, independentemente do número de resultados possíveis no jogo. Para o propósito de enviesar a UCB na PN-MCTS, isso exigiria um número de árvores de prova linear em relação aos resultados possíveis do jogo. Tal abordagem foi proposta em [14], introduzindo uma segunda árvore PN para tratar empates. (Note que manter apenas o resultado atrativo, como na Multiple-Outcome PNS [21], não funciona quando aplicado à busca dinâmica durante o jogo.)

Defendemos que usar uma solução bem estabelecida e mais geral, como a extensão de Limites de Pontuação, é uma opção melhor do que estender ainda mais a PN-MCTS para tarefas sobrepostas de pular subárvores resolvidas e enviesar a seleção final de movimento, como pretendido por [14]. Embora árvores PN possam ser usadas dessa forma, isso é ineficiente e o código resultante é menos gerenciável.

Os resultados (mostrados na Seç. 4.2) confirmam que a GPN-MCTS funciona bem quando combinada com o método de Limites de Pontuação, e que não há perda significativa de qualidade entre os resultados de GPN-MCTS vs. MCTS e de GPN-MCTS com Limites de Pontuação vs. MCTS com Limites de Pontuação.

3.5 O Algoritmo GPN-MCTS

O pseudocódigo de uma única iteração da GPN-MCTS é mostrado no Alg. 1. A estrutura geral do algoritmo segue a definição de MCTS de [7] e usa a mesma terminologia sempre que possível. Por exemplo, DEFAULTPOLICY(s) codifica a simulação padrão do estado de jogo dado até um estado terminal.

A flag *needRecalc* de um nó é a introduzida pela otimização descrita na Seção 3.2.5 e é definida como falsa dentro do procedimento UPDATECHILDRENPNSCORES(). Os detalhes desse procedimento são omitidos, pois ele apenas calcula os valores de viés de seleção dos nós filhos para o jogador da vez no nó, de acordo com as fórmulas da Seção 3.2.

BESTUCTPNCHILD retorna um filho que maximiza o valor da Fórmula UCT-PN (2), com a observação de que outras funções de viés, em vez da PNRank, podem ser codificadas ali.

A função BACKUP(v_l, Δ) consiste em duas partes separadas. Uma é a retropropagação padrão da MCTS. A outra é baseada no procedimento updateAncestors (cf. [12]) para retropropagar os números de prova de forma otimizada (parada antecipada quando nenhuma mudança é detectada).

Por fim, UPDATEPROOFNUMBER(p) atualiza o número de prova em um nó para um dado jogador de acordo com o procedimento setProofAndDisproofNumbers da PNS. As principais diferenças aqui são que atualizamos apenas números de prova e que a distinção entre nós OR/AND se baseia em p ser ou não o jogador que realiza um movimento naquele nó. Um *proofWinner* em um nó folha é *Unknown* se o estado de jogo associado não é terminal ou, caso contrário, o jogador que venceu o jogo.

Algoritmo 1 O Algoritmo GPN-MCTS

```
Entrada: v_0 – nó raiz atual da árvore GPN-MCTS
function GPN-MCTS-ITERATION(v_0)
    v_l ← TREEPOLICY(v_0)
    Δ ← DEFAULTPOLICY(v_l.STATE())
    BACKUP(v_l, Δ)
Entrada: v – nó raiz da árvore GPN-MCTS
function TREEPOLICY(v)
    while v.STATE().ISNOTTERMINAL() do
        if v.needRecalc then v.UPDATECHILDRENPNSCORES()
        if v.ISNOTFULLYEXPANDED() then
            return EXPAND(v)
        else
            v ← v.BESTUCTPNCHILD()
    return v
Entrada: v – último nó da iteração na árvore GPN-MCTS
function EXPAND(v)
    a ← RANDOMELEMENT(v.UNTRIEDMOVES())
    v' ← CREATENODE(v.STATE().APPLY(a))
    for all p ∈ PLAYERS() do v'.UPDATEPROOFNUMBER(p)
    v.ADDNODE(v', a)
    return v'
Entrada: v_l – último nó da iteração na árvore GPN-MCTS
Entrada: Δ – pontuações finais de cada jogador na iteração
function BACKUP(v_l, Δ)
    v ← v_l
    while v ≠ None do
        v.scoreSum ← v.scoreSum + Δ[leaf.player]
        v.iterations ← v.iterations + 1
        v ← v.PARENT()
    for all p ∈ PLAYERS() do
        v ← v_l
        c ← True
        while c and v ≠ None do
            c ← v.UPDATEPROOFNUMBER(p)
            if c then v.needRecalc ← true
            v ← v.PARENT()
Entrada: p – jogador para o qual a atualização é feita
function NODE.UPDATEPROOFNUMBER(p)
    if ISNOTEXPANDED() then
        if proofWinner = Unknown then proofNumber ← 1
        else if proofWinner = p then proofNumber ← 0
        else proofNumber ← ∞
        return True
    oldProof ← proofNumber
    if p = nodePlayer then      ▷ nó OR da PNS
        proofNumber ← ∞
        for all child ∈ CHILDREN() do
            proofNumber ← MIN(proofNumber, child.proofNumber)
    else      ▷ nó AND da PNS
        proofNumber ← 0
        for all child ∈ CHILDREN() do
            proofNumber ← proofNumber + child.proofNumber
    return proofNumber ≠ oldProof
```

4 Experimentos

Os experimentos foram conduzidos usando o sistema de jogos gerais Ludii, que fornece um ambiente para desenvolvedores testarem suas implementações de agentes de jogo geral [19]. Ele foi escolhido porque contém mais de 1.000 jogos descritos em sua linguagem de descrição de jogos e implementações de vários algoritmos e aprimoramentos padrão (como diversas variantes de MCTS), com uma API única e unificada para o desenvolvimento de agentes de IA.

O algoritmo GPN-MCTS apresentado foi implementado como um aprimoramento dos agentes disponíveis no Ludii.[2] Ele é consistente com a última versão publicamente disponível do arcabouço (ver. 1.3.14) e será proposto como um *pull request* ao repositório oficial do projeto[3] após o processo de revisão. Duas versões da GPN-MCTS estão disponíveis: com o aprimoramento Score Bounded e sem ele. Salvo indicação em contrário, os experimentos são realizados jogando a GPN-MCTS com Score Bounded e reutilização de árvore contra a MCTS com Score Bounded e reutilização de árvore. Para ambos os agentes, o parâmetro C da MCTS é definido como $\sqrt{2}$. As posições dos jogadores são trocadas em todos os testes, de modo que os agentes joguem com ambos os lados igual número de vezes, e os empates contam como meia vitória. Os experimentos foram executados em máquinas diferentes; porém, para qualquer jogo, todos os resultados referentes a ele foram calculados no mesmo hardware, o que os torna comparáveis. Para cada resultado, quando margens de erro são apresentadas, elas representam um intervalo de confiança de 95%.

4.1 Fórmulas de viés UCT-PN e Cpn

Os experimentos principais foram conduzidos em um conjunto de jogos de tabuleiro de soma zero para dois jogadores, incluindo os jogos usados nos experimentos de [14]. Nosso objetivo é testar o comportamento de cada variante de fórmula de viés (PNRank, PNMax, PNSum) e a influência da constante C_{pn} nas melhorias em relação ao agente de linha de base. Testamos $C_{pn} \in \{0{,}0;\ 0{,}1;\ 0{,}5;\ 1{,}0;\ 2{,}0;\ 5{,}0\}$ para manter a consistência com pesquisas anteriores sobre a PN-MCTS. Embora esse intervalo de parâmetros possa ser insuficiente para fornecer a taxa de vitória máxima exata disponível para um dado jogo e variante UCT-PN, ele fornece uma boa estimativa dela, bem como do comportamento geral da função de taxa de vitória para essas configurações. Cada teste consiste em 500 partidas com 0,5 segundo por jogada. Os resultados são apresentados na Tabela 1.

A GPN-MCTS alcançou 80% de taxa de vitória contra a MCTS com Score Bounded em 8 dos 11 jogos de tabuleiro testados, chegando a 90% em dois casos (Ataxx e Lines of Action). Para os demais jogos testados, os resultados também são vitórias confiáveis, com a menor melhor pontuação, 63,2%, obtida para Knighttrough.

A primeira observação é que os melhores valores de C_{pn} diferem muito entre os jogos. Para alguns jogos, o melhor valor foi o menor testado e, para outros, o maior (o que sugere que valores de C_{pn} ainda melhores podem potencialmente ser encontrados fora do intervalo testado). Além disso, na maioria dos jogos, a variação da taxa de vitória entre a melhor e a pior escolha de C_{pn} é enorme (no caso extremo de Lines of Action 8 × 8, a taxa de vitória caiu de 82,5% com $C_{pn} = 1$ para 25,9% com $C_{pn} = 5$). Esse é um forte indicador de que escolher um valor que se comporte melhor na média não é uma boa ideia, e os resultados obtidos dessa forma podem estar muito longe dos ótimos.

A segunda observação baseia-se na comparação do comportamento e dos resultados das três fórmulas de viés testadas. Não há um vencedor claro sobre qual das fórmulas é a melhor. Especialmente porque, muitas vezes, as taxas de vitória máximas não estão distantes entre si, e o ajuste local de C_{pn} pode possivelmente inverter os resultados. (E, às vezes, especialmente para a PNSum, que tem tendência a ter picos em valores maiores de C_{pn}, o pico pode estar ainda mais além do intervalo de parâmetros testado.) De modo geral, a PNRank parece uma escolha segura, obtendo os melhores resultados, ou resultados relativamente próximos dos melhores, na maioria dos jogos. Em alguns casos, como no Minishogi, essa fórmula de viés mostra uma vantagem clara sobre as demais. Em outros, por exemplo no Surakarta, a PNMax é claramente melhor, com mais de 20 pontos percentuais de vantagem. Embora para muitos jogos a PNSum tenha resultados inferiores, também há casos (Reversi) em que ela tem o melhor desempenho entre as três.

Em resumo, se o jogo é suscetível à exploração baseada em PN, então usar qualquer uma das fórmulas deve levar a uma melhoria, mas o valor real dessa melhoria depende da escolha específica e pode diferir significativamente.

[2] O código-fonte está disponível no apêndice e será tornado público após o processo de revisão.

[3] https://github.com/Ludeme/Ludii

4.2 Influência do Score Bounded na GPN-MCTS

Para alguns dos jogos e para a fórmula PNRank, executamos experimentos sem o aprimoramento Score Bounded (tanto para a GPN-MCTS quanto para a MCTS). Assim, podemos analisar se o método baseado em PN funciona tão bem no algoritmo puro quanto em união com outras melhorias. O problema potencial de qualquer melhoria em qualquer algoritmo de IA complexo é que, embora funcione isoladamente, seus benefícios se degradam quando aplicada junto com outras melhorias. Além disso, em cenários mais próximos do mundo real, não podemos esperar que nosso oponente seja uma implementação básica, de modo que os aprimoramentos devem ser capazes de mostrar melhorias também contra um oponente mais avançado.

Os resultados de nossa comparação mostram que, em geral, aprimorar tanto a GPN-MCTS quanto o oponente com a extensão Score Bounded mantém as taxas de vitória em faixas semelhantes. Note, porém, que a PN-MCTS, cujo objetivo central é enviesar a exploração em direção a nós potencialmente rápidos de provar, é especialmente adequada para funcionar bem com a extensão Score Bounded. Isso pode explicar por que, com frequência, os resultados que incluem SB tendem a ser (ligeiramente) melhores do que os da MCTS pura.

4.3 Sobrecarga

A necessidade de manter estruturas relacionadas à PNS além da árvore MCTS padrão implica que a PN-MCTS costuma ser mais lenta que a MCTS pura. Assim, ela geralmente realizará menos iterações em qualquer orçamento de tempo dado. Por essa razão, todos os experimentos deste artigo usam orçamentos baseados em tempo, o que consideramos uma comparação justa, já que orçamentos baseados em simulações ignoram o fator de sobrecarga computacional. A GPN-MCTS é desenvolvida como uma extensão da implementação de MCTS fornecida pelo sistema Ludii, para garantir que qualquer diferença de desempenho se deva exclusivamente à implementação do aprimoramento proposto.

Para medir a sobrecarga, incluímos $C_{pn} = 0$ nos resultados da Tabela 1. Essa taxa de vitória representa a partida entre dois algoritmos MCTS com Score Bounded (em termos de comportamento), mas com um deles gastando tempo de computação adicional para gerenciar números de prova e valores UCT-PN. A maioria dessas taxas de vitória está dentro da distância do intervalo de confiança em relação a 50%, o que significa que a sobrecarga computacional da GPN-MCTS parece ser pequena o suficiente para não afetar os resultados esperados.

Tabela 1: Resultados da GPN-MCTS versus MCTS. A variante que inclui SB significa que ambos os algoritmos usam Score Bounded e reutilização de árvore; caso contrário, é apenas reutilização de árvore. (500 partidas, 1 s por jogada.) Valores em % de vitórias, com intervalo de confiança de 95%.

Variante	Valor de Cpn					
	0,0	0,1	0,5	1,0	2,0	5,0
Ataxx						
PNRank	45,9 ± 4,37	58,0 ± 4,32	82,3 ± 3,33	87,0 ± 2,95	89,4 ± 2,70	89,5 ± 2,67
PNRank+SB	50,1 ± 4,38	60,6 ± 4,27	85,3 ± 3,09	89,2 ± 2,72	91,4 ± 2,46	92,0 ± 2,36
PNMax+SB	49,2 ± 4,38	65,3 ± 4,17	92,4 ± 2,33	92,3 ± 2,33	91,0 ± 2,51	89,1 ± 2,71
PNSum+SB	45,6 ± 4,36	56,5 ± 4,33	55,7 ± 4,34	56,2 ± 4,34	61,6 ± 4,25	71,2 ± 3,93
Awari						
PNRank+SB	48,1 ± 3,97	71,0 ± 3,53	74,3 ± 3,32	62,2 ± 3,63	50,4 ± 3,92	43,2 ± 4,21
PNMax+SB	50,9 ± 3,95	78,2 ± 3,21	69,8 ± 3,50	56,3 ± 4,28	45,5 ± 4,27	45,1 ± 4,25
PNSum+SB	49,8 ± 4,04	62,2 ± 3,73	78,8 ± 3,28	79,7 ± 3,07	77,2 ± 3,34	67,9 ± 3,77
Knightthrough						
PNRank	48,0 ± 4,38	50,6 ± 4,39	52,2 ± 4,38	52,8 ± 4,38	62,6 ± 4,25	74,0 ± 3,85
PNRank+SB	53,2 ± 4,38	45,8 ± 4,37	46,2 ± 4,37	56,3 ± 4,33	54,8 ± 4,37	59,2 ± 4,31
PNMax+SB	46,8 ± 4,38	56,6 ± 4,35	50,8 ± 4,39	62,8 ± 4,24	63,2 ± 4,21	54,4 ± 4,37
PNSum+SB	49,0 ± 4,39	50,0 ± 4,39	56,0 ± 4,36	52,0 ± 4,38	55,8 ± 4,36	60,0 ± 4,30
Lines of Action (7 × 7)						
PNRank	52,1 ± 4,38	67,4 ± 4,11	85,0 ± 3,13	87,0 ± 2,95	88,4 ± 2,81	80,2 ± 3,49
PNRank+SB	47,6 ± 4,37	67,1 ± 4,12	82,8 ± 3,31	91,3 ± 2,47	91,8 ± 2,39	83,6 ± 3,25
PNMax+SB	52,6 ± 4,38	66,3 ± 4,14	81,3 ± 3,42	84,3 ± 3,14	68,6 ± 4,06	65,8 ± 4,13
PNSum+SB	54,6 ± 4,37	55,1 ± 4,36	60,9 ± 4,28	64,9 ± 4,18	67,9 ± 4,08	69,3 ± 4,04
Lines of Action (8 × 8)						
PNRank+SB	51,8 ± 4,38	63,6 ± 4,22	83,8 ± 3,23	87,4 ± 2,91	81,9 ± 3,37	55,2 ± 4,35
PNMax+SB	50,8 ± 4,39	55,8 ± 4,36	68,8 ± 4,07	82,5 ± 3,33	63,7 ± 4,21	25,9 ± 3,83

Variante	Valor de Cpn					
	0,0	**0,1**	**0,5**	**1,0**	**2,0**	**5,0**
PNSum+SB	48,0 ± 4,38	54,5 ± 4,33	52,4 ± 4,38	55,6 ± 4,41	55,5 ± 4,75	64,3 ± 4,48
Los Alamos chess						
PNRank	53,2 ± 4,10	71,9 ± 3,76	85,5 ± 2,94	82,4 ± 3,22	80,5 ± 3,36	80,5 ± 3,39
PNRank+SB	47,3 ± 4,16	71,7 ± 3,73	81,0 ± 3,33	84,5 ± 3,02	80,5 ± 3,38	79,3 ± 3,43
PNMax+SB	48,7 ± 4,16	71,1 ± 3,74	73,4 ± 3,73	76,5 ± 3,52	80,5 ± 3,35	78,7 ± 3,45
PNSum+SB	49,7 ± 4,11	60,3 ± 4,10	64,5 ± 3,95	72,7 ± 3,75	77,5 ± 3,46	77,0 ± 3,54
Minishogi						
PNRank	47,4 ± 4,38	55,0 ± 4,37	67,8 ± 4,17	69,2 ± 4,05	56,8 ± 4,35	45,6 ± 4,37
PNRank+SB	45,2 ± 4,37	54,4 ± 4,37	67,8 ± 4,10	66,2 ± 4,15	63,8 ± 4,22	46,0 ± 4,37
PNMax+SB	52,4 ± 4,38	49,6 ± 4,39	51,2 ± 4,39	34,6 ± 4,17	31,6 ± 4,08	36,2 ± 4,22
PNSum+SB	49,6 ± 4,39	52,4 ± 4,38	57,0 ± 4,34	59,4 ± 4,31	51,8 ± 4,38	48,8 ± 4,39
Reach Chess						
PNRank	44,4 ± 4,36	58,6 ± 4,32	64,8 ± 4,19	66,2 ± 4,15	71,8 ± 3,95	76,8 ± 3,70
PNRank+SB	50,2 ± 4,39	62,0 ± 4,26	64,6 ± 4,20	72,2 ± 3,93	79,0 ± 3,57	78,0 ± 3,64
PNMax+SB	49,2 ± 4,39	58,8 ± 4,32	70,2 ± 4,01	65,2 ± 4,18	69,8 ± 4,03	55,2 ± 4,36
PNSum+SB	46,2 ± 4,37	57,4 ± 4,34	59,8 ± 4,30	63,6 ± 4,22	64,4 ± 4,20	68,2 ± 4,09
Reversi						
PNRank	48,7 ± 4,28	65,1 ± 4,11	71,6 ± 3,89	49,9 ± 4,32	43,4 ± 4,33	42,5 ± 4,32
PNRank+SB	50,0 ± 4,32	71,0 ± 3,87	69,4 ± 3,99	53,2 ± 4,34	42,4 ± 4,27	37,2 ± 4,22
PNMax+SB	50,0 ± 4,24	63,1 ± 4,16	59,0 ± 4,26	45,2 ± 4,30	36,2 ± 4,18	35,0 ± 4,15
PNSum+SB	49,5 ± 4,28	55,0 ± 4,27	60,7 ± 4,21	71,7 ± 3,86	76,7 ± 3,64	64,1 ± 4,14
Skirmish						
PNRank+SB	50,6 ± 4,16	62,4 ± 4,07	62,3 ± 4,11	61,7 ± 4,17	60,8 ± 4,17	55,8 ± 4,29
PNMax+SB	51,5 ± 4,20	65,1 ± 4,00	65,3 ± 4,05	58,0 ± 4,17	56,2 ± 4,26	39,0 ± 4,18
PNSum+SB	50,5 ± 4,23	49,9 ± 4,19	56,4 ± 4,11	55,3 ± 4,12	63,4 ± 4,38	63,8 ± 4,03
Surakarta						
PNRank+SB	49,5 ± 4,34	60,5 ± 4,23	60,6 ± 4,25	59,1 ± 4,26	46,7 ± 4,33	15,2 ± 3,11
PNMax+SB	48,6 ± 4,36	65,1 ± 4,13	82,0 ± 3,33	68,4 ± 4,05	44,4 ± 4,32	22,4 ± 3,67
PNSum+SB	47,8 ± 4,35	52,5 ± 4,63	54,3 ± 4,50	50,9 ± 4,35	54,6 ± 4,34	57,7 ± 4,27

5 Conclusão

Os resultados experimentais mostram que a GPN-MCTS é um aprimoramento da MCTS bastante impactante, e defendemos que ela tem todos os motivos para ser considerada uma melhoria básica a ser implementada ao lado do Solver/MAST/RAVE e de outros desenvolvimentos clássicos.

Ela é relativamente simples de implementar, baseada em um algoritmo clássico e bem descrito. Combina-se com o aprimoramento obrigatório da MCTS com Limites de Pontuação. É fácil testar se sua aplicação será benéfica para um dado jogo — comparar um algoritmo sem a extensão com qualquer fórmula de viés e um ou dois valores pequenos de C_{pn} deve dar a indicação correta. E os ganhos potenciais podem variar de uma vitória confiável de 60% a uma vitória decisiva de 90%.

Neste artigo, concentramos nossos experimentos em jogos para dois jogadores, apesar de a GPN-MCTS poder, de fato, lidar com jogos com mais jogadores. Embora os jogos multijogador sejam um desafio interessante, até agora a integração de capacidades de resolução na MCTS levou apenas a melhorias limitadas [18]. Por isso, consideramos a aplicação da GPN-MCTS nesses domínios como pesquisa futura.

Nossos experimentos preliminares indicaram que há jogos nos quais a extensão GPN não causa impacto visível (Pentago, Pentalath) ou até piora os resultados (Connect Four, Diagonals). Isso era de se esperar, pois a Busca por Números de Prova nem sempre funciona quando aplicada de maneira agnóstica ao jogo. A PNS aproveita situações em que há caminhos vencedores profundos e estreitos. Em alguns jogos, é necessário introduzir conhecimento de domínio [12] para moldar a árvore de busca de modo que surjam caminhos estreitos e forçados. Testar se é possível transferir esse conhecimento para o arcabouço GPN-MCTS e se isso influenciará positivamente os resultados nesses jogos é um dos caminhos promissores para trabalhos futuros.

Agradecimentos

Este artigo é baseado no trabalho da COST Action CA22145 – GameTable, apoiada pela COST (European Cooperation in Science and Technology).

Esta pesquisa foi apoiada em parte pelo National Science Centre, Polônia, sob o número de projeto 2021/41/B/ST6/03691 (Jakub Kowalski).

Referências

[1] L. V. Allis, M. van der Meulen, and H. J. van den Herik. Proof-Number Search. Artificial Intelligence, 66(1):91–123, 1994.

[2] B. Arneson, R. B. Hayward, and P. Henderson. Monte Carlo Tree Search in Hex. IEEE Transactions on Computational Intelligence and AI in Games, 2(4):251–258, 2010.

[3] P. Auer, N. Cesa-Bianchi, and P. Fischer. Finite-Time Analysis of the Multiarmed Bandit Problem. Machine Learning, 47(2–3):235–256, 2002.

[4] H. Baier and M. H. M. Winands. MCTS-Minimax Hybrids. IEEE Transactions on Computational Intelligence and AI in Games, 7(2):167–179, 2015. doi: 10.1109/TCIAIG.2014.2366555.

[5] Y. Björnsson and H. Finnsson. CadiaPlayer: A Simulation-Based General Game Player. IEEE Transactions on Computational Intelligence and AI in Games, 1(1):4–15, 2009.

[6] D. M. Breuker, J. W. H. M. Uiterwijk, and H. J. van den Herik. The PN2-search algorithm. In H. J. van den Herik and B. Monien, editors, Advances in Computer Games 9, pages 115–132. Universiteit Maastricht, Maastricht, The Netherlands, 2001.

[7] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton. A Survey of Monte Carlo Tree Search Methods. IEEE Transactions on Computational Intelligence and AI in Games, 4(1):1–43, 2012.

[8] T. Cazenave and A. Saffidine. Score Bounded Monte-Carlo Tree Search. In Computers and Games, volume 6515 of LNCS, pages 93–104, 2011.

[9] G. M. J.-B. Chaslot, M. H. M. Winands, H. J. van den Herik, J. W. H. M. Uiterwijk, and B. Bouzy. Progressive Strategies for Monte-Carlo Tree Search. New Mathematics and Natural Computation, 4(3):343–357, 2008.

[10] R. Coulom. Efficient Selectivity and Backup Operators in Monte-Carlo Tree Search. In Computers and Games (CG 2006), volume 4630 of Lecture Notes in Computer Science, pages 72–83, 2007.

[11] E. Doe, M. H. M. Winands, D. J. N. J. Soemers, and C. Browne. Combining Monte-Carlo tree search with proof-number search. In Proceedings of the 2022 IEEE Conference on Games, pages 206–212, 2022.

[12] A. Kishimoto, M. H. M. Winands, M. Müller, and J.-T. Saito. Game-Tree Search Using Proof Numbers: The First Twenty Years. ICGA Journal, 35(3):131–156, 2012.

[13] L. Kocsis and C. Szepesvári. Bandit based Monte-Carlo planning. In J. Fürnkranz, T. Scheffer, and M. Spiliopoulou, editors, Machine Learning: ECML 2006, volume 4212 of Lecture Notes in Computer Science, pages 282–293. Springer, Berlin, Heidelberg, 2006.

[14] J. Kowalski, E. Doe, M. H. M. Winands, D. Górski, and D. J. N. J. Soemers. Proof Number Based Monte-Carlo Tree Search. IEEE Transactions on Games, 17(1):148–157, 2024.

[15] M. Lanctot, M. H. M. Winands, T. Pepels, and N. R. Sturtevant. Monte Carlo Tree Search with Heuristic Evaluations using Implicit Minimax Backups. In 2014 IEEE Conference on Computational Intelligence and Games, CIG 2014, pages 341–348, 2014.

[16] R. J. Lorentz. Amazons Discover Monte-Carlo. In Computers and Games (CG 2008), volume 5131 of Lecture Notes in Computer Science, pages 13–24, 2008.

[17] A. Nagai. A new AND/OR tree search algorithm using proof number and disproof number. In Proceedings of Complex Games Lab Workshop, pages 40–45. ETL, Tsukuba, Japan, 1998.

[18] J. A. M. Nijssen and M. H. M. Winands. Enhancements for multi-player Monte-Carlo tree search. In H. J. van den Herik, H. Iida, and A. Plaat, editors, Computers and Games (CG 2010), volume 6515 of Lecture Notes in Computer Science, pages 238–249. Springer Berlin Heidelberg, 2011.

[19] É. Piette, D. J. N. J. Soemers, M. Stephenson, C. F. Sironi, M. H. M. Winands, and C. Browne. Ludii – the ludemic general game system. In G. D. Giacomo, A. Catala, B. Dilkina, M. Milano, S. Barro, A. Bugarín, and J. Lang, editors, Proceedings of the 24th European Conference on Artificial Intelligence (ECAI 2020), volume 325 of Frontiers in Artificial Intelligence and Applications, pages 411–418. IOS Press, 2020.

[20] O. Randall, M. Müller, T.-H. Wei, and R. Hayward. Expected work search: Combining win rate and proof size estimation. In Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI ’24, pages 7003–7011, 2024.

[21] A. Saffidine and T. Cazenave. Multiple-outcome Proof Number Search. In ECAI, pages 708–713. 2012.

[22] J.-T. Saito and M. H. M. Winands. Paranoid Proof-Number Search. In Proceedings of the Computational Intelligence and Games Conference (CIG'10), pages 203–210, 2010.

[23] D. Silver, J. Schrittwieser, K. Simonyan, I. Antonoglou, A. Huang, A. Guez, T. Hubert, L. Baker, M. Lai, A. Bolton, Y. Chen, T. Lillicrap, F. Hui, L. Sifre, G. van den Driessche, T. Graepel, and D. Hassabis. Mastering the Game of Go Without Human Knowledge. Nature, 550:354–359, 2017.

[24] H. J. van den Herik and M. H. M. Winands. Proof-Number Search and Its Variants. In Oppositional Concepts in Computational Intelligence, pages 91–118. 2008. ISBN 978-3-540-70829-2.

[25] M. H. M. Winands and Y. Björnsson. αβ-based Play-outs in Monte-Carlo Tree Search. In 2011 IEEE Conference on Computational Intelligence and Games (CIG 2011), pages 110–117. IEEE, 2011.

[26] M. H. M. Winands, Y. Björnsson, and J.-T. Saito. Monte-Carlo Tree Search Solver. In Computers and Games (CG 2008), volume 5131 of Lecture Notes in Computer Science (LNCS), pages 25–36, 2008.

[27] M. H. M. Winands, Y. Björnsson, and J.-T. Saito. Monte Carlo Tree Search in Lines of Action. IEEE Transactions on Computational Intelligence and AI in Games, 2(4):239–250, 2010.

[28] M. Świechowski, K. Godlewski, B. Sawicki, and J. Mańdziuk. Monte Carlo Tree Search: A review of recent modifications and applications. Artificial Intelligence Review, 56(3):2497–2562, 2023. doi: 10.1007/S10462-022-10228-Y.